KNAPSACK PROBLEM:

problem Defenition:

Given with a set of 'n' items, where each item is associated with two attributes weight (w) and value (v), and a knapsack (bag) of capacity 'W'. The problem is to find the subset of items that fits into the knapsack with their sum of weights < 'W', and providing a maximum profit (sum of values is maximum).

Solution using BB technique:

→ The state space tree to be constructed will be a binary tree, that is at each level, we will decide to either put the item into knapsack or not.

→ The path from to root to any node will represent the subset of items included or not.

→ Left Branch indicates: include the item into knapsack

→ Right Branch indicates: item not included.

STEP 1:

Find the value/weight ratio of each item.

Eg:

Item	Weight	Value	Value/Weight
1	5	40	8
2	7	35	5
3	2	18	9
4	4	4	1
5	5	10	2
6	1	2	2

W=15

Scanned by CamScanner

STEP2: Arrange the items in decending order based on value/weight.

Item	Weight	Value	Value/weight
1 row) This is for 1st item	3	2	18	9
2nd item	1	5	40	8
3rd item	2	7	35	5
4th item	5	5	10	2
5th item	6	1	2	2
6th item	4	4	4	1

for Calculation perspective

Now this is the order of items in which they have to be decided for inclusion into knapsack.

STEP3: Determine the Bound value (Upper Bound) based on the following formula.

$$UB = v + (W - w)(V_{i+1}/W_{i+1})$$

Find the Root Node bound value as follows, Whe no items are included, $w = 0, v = 0$

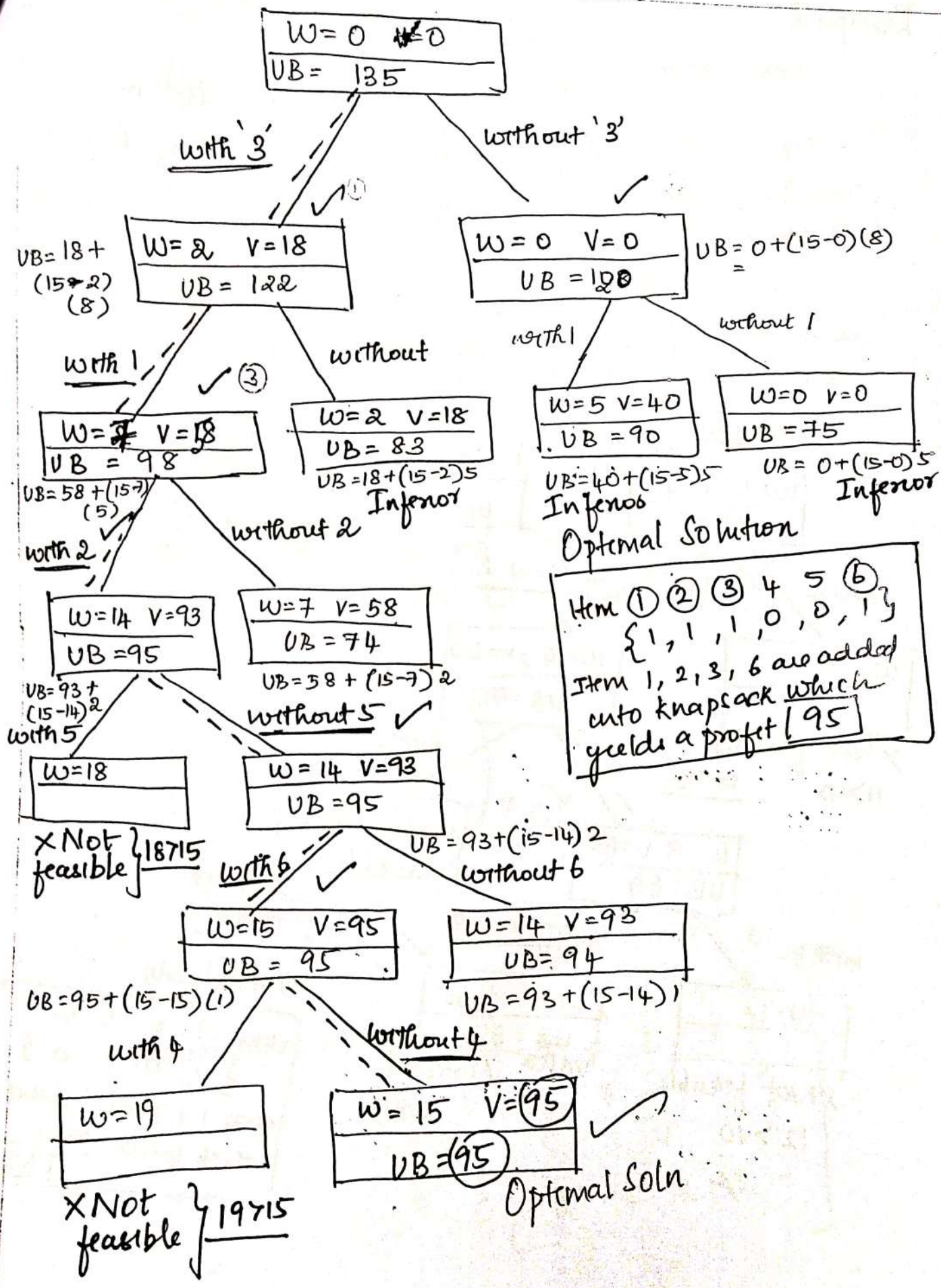
hence:

$$UB = 0 + (15 - 0)(V_1/W_1)$$

$$= 0 + (15 - 0)\,9$$

$$UB = 135$$

Scanned by CamScanner

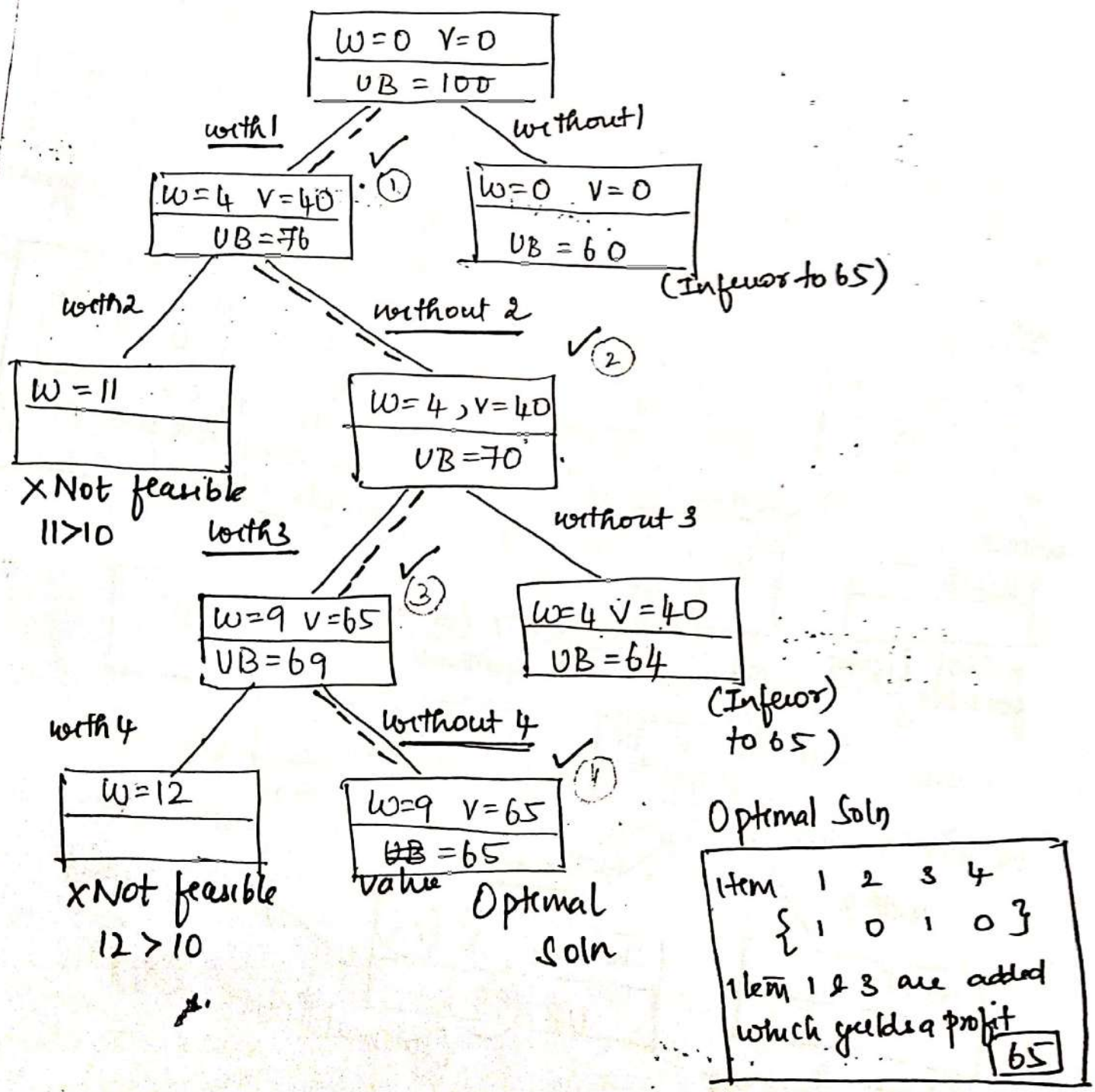


Scanned by CamScanner

Example 2:

Item	weight	value	value/weight
1	4	40	10
2	7	42	6
3	5	25	5
4	3	12	4

} Already in descending Order.



Scanned by CamScanner

BACK TRACKING

Introduction:

* Backtracking is a design technique used to solve some large instances of difficult combinatorial problem. It is considered as an improvement over exhaustive search.

* principle idea of backtracking:

The idea of backtracking is to construct solutions one component at a time & evaluate such partially constructed solutions as follows,

↳ If the partially constructed solution can be developed further without violating the problem's constraints, it is done by taking the first remaining legitimate option for the next component.

↳ If there is no legitimate option for the next component, the algorithm backtracks to replace the last component of the partially constructed solution with its next option.

* The idea of backtracking is implemented by constructing a tree of choices been made called state-space tree.

Scanned by CamScanner

* If the current node turns out to be non-promising the algorithm backtracks to the node parent to consider the next possible option for its last component. If there is no such option it backtracks one more level up the tree and so on.

Problems to be solved using Backtracking Algorithm:

→ n-Queen's problem
→ Hamiltanian Circuit
→ Sum of Subsets.

n-Queen's Problem:

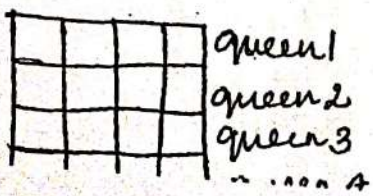
Problem Definition:

The problem is to place 'n' queens on an n-by-n chessboard so that no two queens attack each other by being in the same row or in the same column or on the same diagnal.

Solving the problem using backtracking:

* When n = 1, 1-Queen Problem, the problem has a trivial solution.
* When n = 2, 3, the problem has no solution
* So let us consider the four-Queen problem and solve its using backtracking.
* 4-Queen Problem:

→ Assume Queen 1 to be placed in row 1,
Queen 2 " " " row 2,
" " " " row 3,



Scanned by CamScanner

* Solution Representation:

→ Since each of the queen is to placed in their respective row, assigning the correct column for each each queen in their respective row gives the solution to the problem.

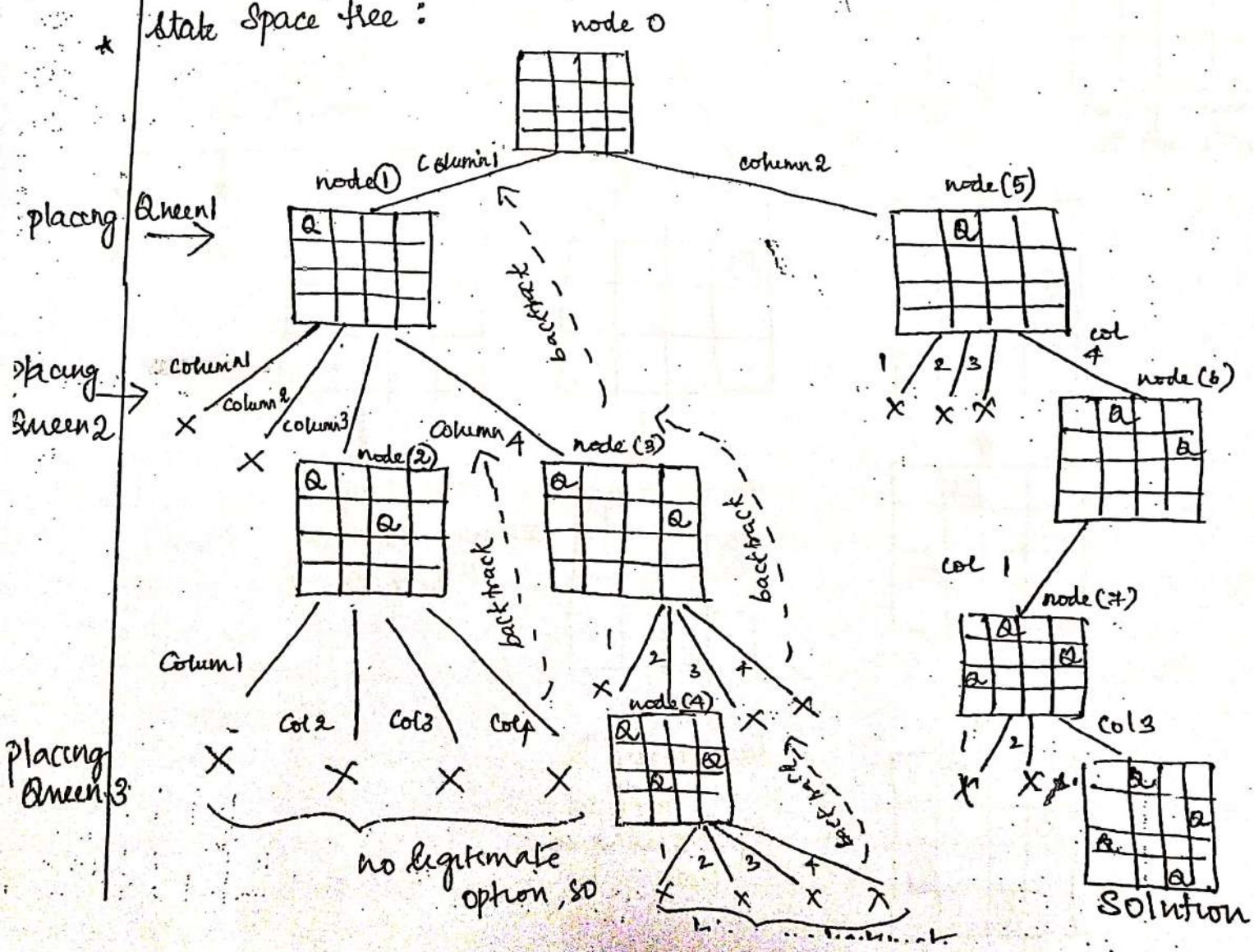
→ That is, if we represent the solution in the form of a solution vector, $(x_1, x_2, x_3 \ldots, x_n)$

→ x_1 – represents the column in which queen 1 is placed

→ x_2 – represents the column in which queen 2 is placed.

→ x_n – represents the column in which queen 'n' is placed.

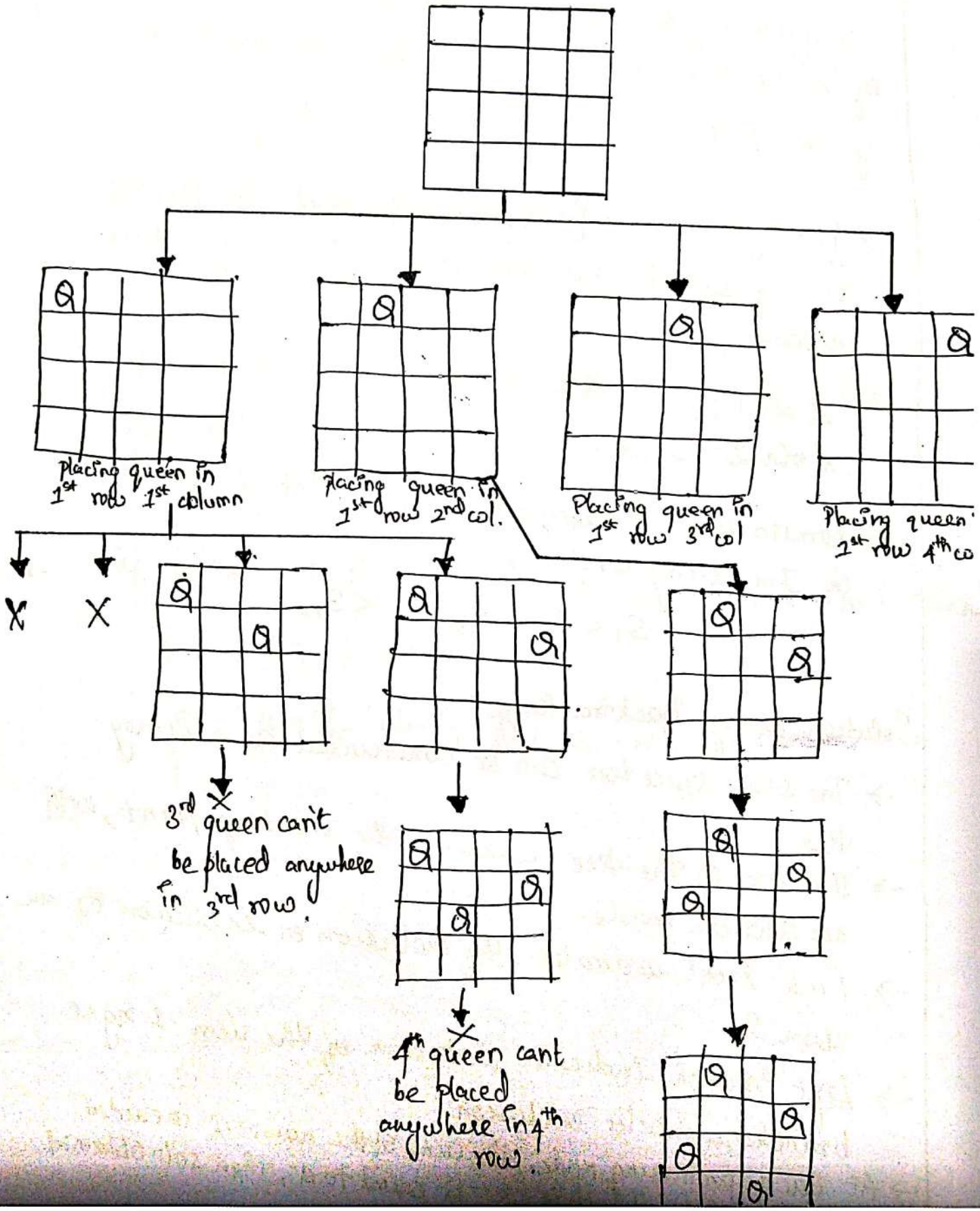
* State Space tree:



Scanned by CamScanner

n-QUEEN'S PROBLEM:

If n=4. State space tree of solving the four-queen problem by backtracking.



Scanned by CamScanner

Given $S = \{3, 5, 6, 2\}$ and $m = 11$.

Solution:

Arrange the elements in increasing order.

$S = \{2, 3, 5, 6\}$

Scanned by CamScanner

HAMILTONIAN CIRCUIT PROBLEM:

Problem Defenition:

Given a graph/network G, the problem is to find a Hamiltonian circuit which starts from a vertex, visits all the other vertices exactly once and reaches the start vertex again, forming a cycle.

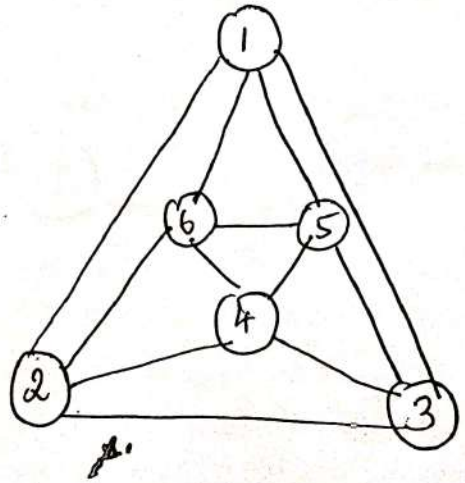
Solution using Backtracking:

→ Consider a vertex as source vertex.

→ Find its adjacent vertices, & select the next legitemate option & explore that vertex.

→ Continue the above step until all vertices are visited & we reach back source vertex, if not backtrack and replace a legitemate option of the previous component (vertex).

Example:

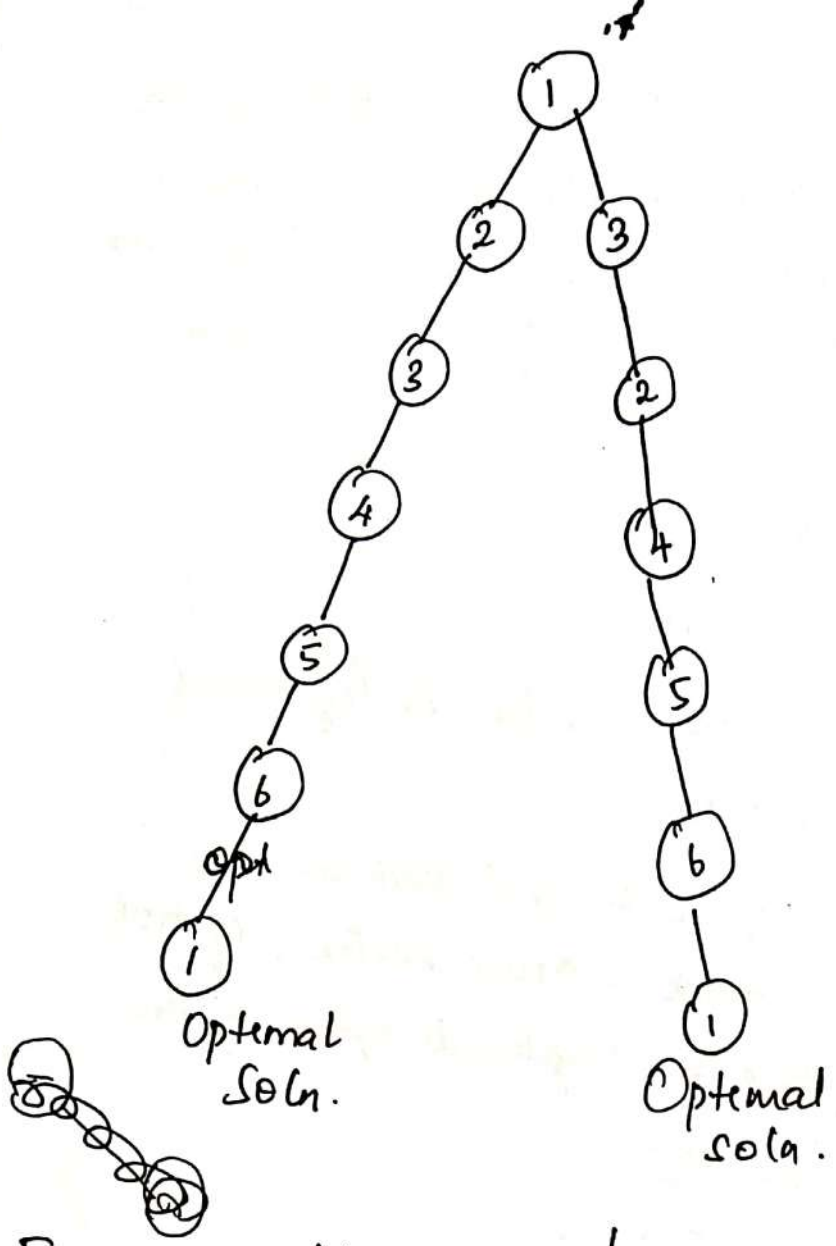


→ Consider '1' as Source Vertex

Adjacency Matrix

	1	2	3	4	5	6
1	0	1	1	0	1	1
2	1	0	1	1	0	1
3	1	1	0	1	1	0
4	0	1	1	0	1	1
5	1	0	1	1	0	1
6	1	1	0	1	1	0

Scanned by CamScanner



For the problem many Hamiltonian Circuit can be constructed.

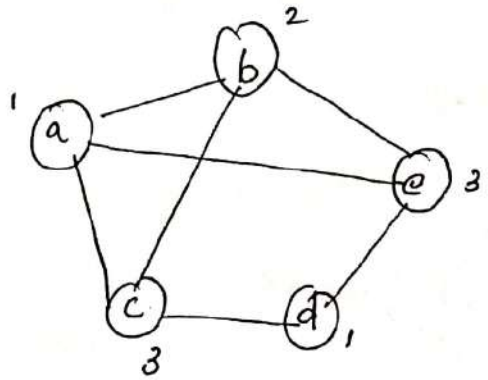
Scanned by CamScanner

Graph Coloring:

Defination:

Given with a Graph G, the m-colorability optimization problem is to find the smallest integer 'm' (no of colors). for which the graph can be colored with a constraint that no two adjacent nodes have the same color.

Eg:



* The integer 'm' is refered as the chromatic number. The above graph G is, chromatic number 'm' = 3. This graph can be colored with 3 colors.

* Note: If a graph has an degree 'd', then it can be maximum colored with d+1 colors.

Solving using Backtracking:

Based on backtracking, the following steps are to be followed to perform valid Coloring of a graph G = (V, E).

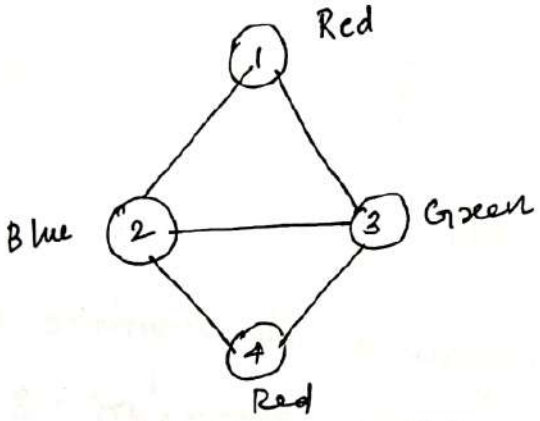
1. Order the nodes arbitrarily
2. Assign the first node a color.
3. Given the partial assignment of colors $(c_1, c_2 \ldots c_{i-1})$ to the first i-1 nodes, try to find a color for the ith node in the graph.

Scanned by CamScanner

4. If there is no possible color for the ith node gu... the previous choices backtrack to the previous soln.

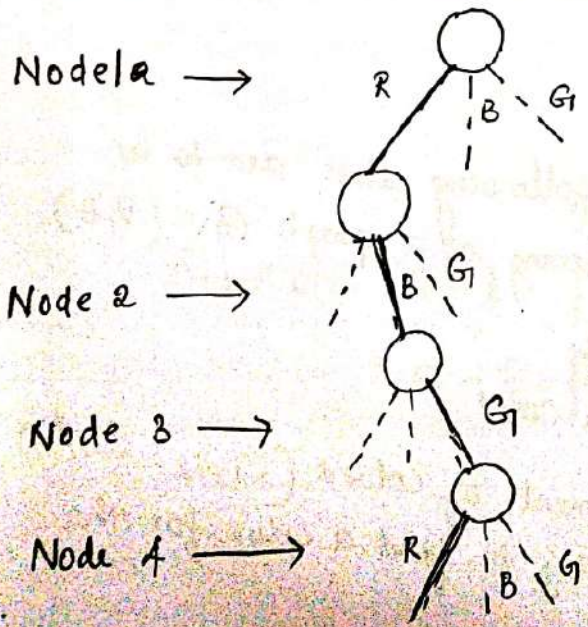
Example 1:

Consider a Graph $G_1 = (V, E)$ where $V = \{1, 2, 3, 4\}$ and $E = \{(1,2), (1,3), (2,3), (2,4), (3,4)\}$ and let $m = 3$. These are 3 Find the valid Colorings of Graph G_1.



Solution:

1. Order of the nodes : ~~a,b,c,e,d~~ 1, 2, 3, 4
2. $m = 3$, let it be Red(R), Blue (B) and Green (G)



Finally if we search the entire state space tree, there will be six valid Coloring for Graph G_1.

node	Soln 1	2	3	4	5	6
1	R	R	B	B	G	G
2	B	G	R	G	R	B
3	G	B	G	R	B	R
4	R	R	B	B	G	G

Scanned by CamScanner

Example 2:

Consider the following states of India,

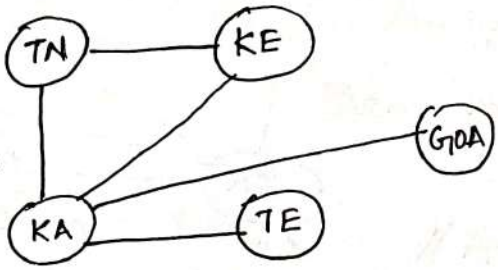
V = { TamilNadu, Kerala, Karnataka, Goa, Telengana }

Color the states in such a way that no two adjacent states should have the same Color.

Check is it possible to color with 2 colors or 3 colors?.

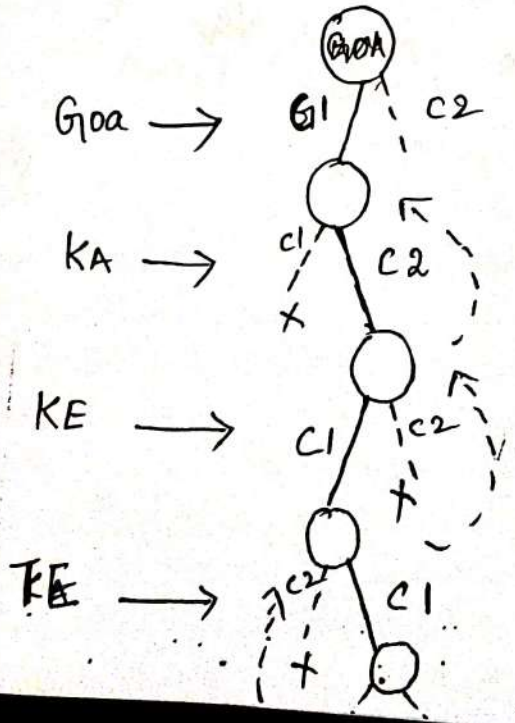
Solution:

1. Representing the problem as graph - Coloring Problem.



2. Ordering the nodes alphabetically: GOA, KA, KE, TE, TN

3. <u>when m = 2</u>, let it be C1, C2



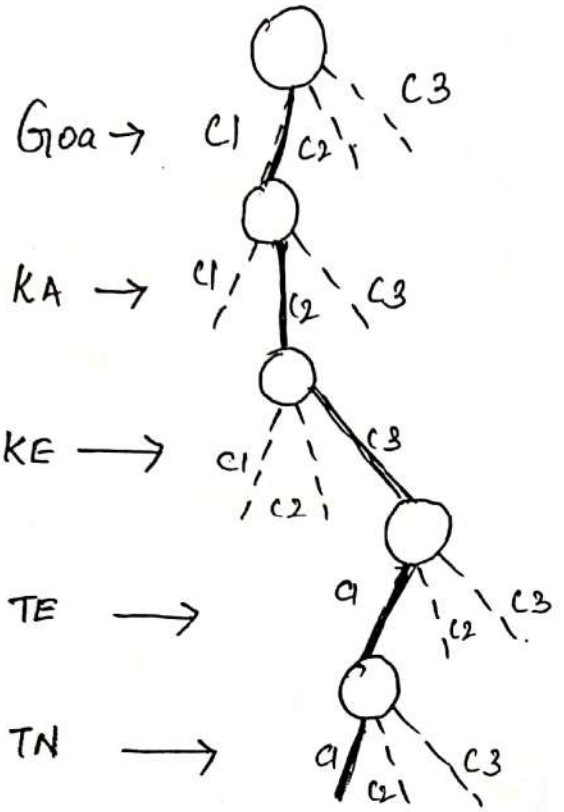
If we try with C1 for Goa and work it out we end up again with a dead end, hence,

m=2 is not possible

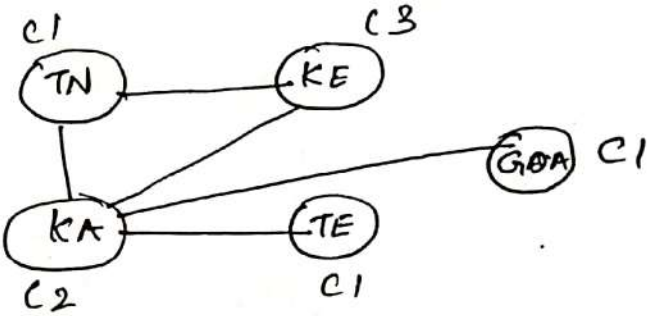
2 colors cannot be used for a valid Coloring to the Graph G.

Scanned by CamScanner

When $m = 3$, let it be C1, C2 & C3



GOA – C1
KA – C2
KE – C3
TE – C1
TN – C1



III[rd]y we can have other valid Colorings too.

Hence for the above problem $m = 3$, that is we need 3 Colors to color the Map in such a way that no two adjacent state have the same Color.

Scanned by CamScanner

```
Algorithm mColoring (k)
// The graph is represented by its boolean
   adjacency matrix G[1:n, 1:n].
// All assignments of 1, 2, ..., m to the vertice
   of the graph such that the adjacent vertices
   are assigned distinct integers.
// k is the index of next vertex to color.
{
   repeat
   {
      // Generate all legal assignments for x[k].
      NextValue (k); // Assign to x[k] a legal color
      if (x[k] == 0) then return; // no new possible
                                     colour
      if (k == n) then          // Atmost m colors have
                                   been used to color n vertice
         write (x[1:n]);
      else
         mColoring (k+1);
   }
   until (false);
}.
```

Scanned by CamScanner

Algorithm NextValue (k)

// x[1] ... x[k-1] have been assigned integer values in the range [1, m] such that adjacent vertices have distinct integers.

// A value for x[k] is determined in the range [0, m]

// x[k] is assigned the next highest numbered color maintaining distinctness from adjacent vertices of vertex k. If no such colour exists, then x[k] is 0.

```
{
  repeat
  {
    x[k] = (x[k]+1) mod (m+1); // next highest colour
    if (x[k] = 0) then return; // All colours have been used.
    for (j := 1 to n do
    {
      if ((G[k,j] ≠ 0) and (x[k] = x[j]))
      // if (k,j) is an edge and if adjacent vertices have same colour.
      then break;
    }
    if (j = n+1) then return // new colour found
  }
  until (false);
}
```

BRANCH

Scanned by CamScanner

BRANCH AND BOUND:

→ Branch and bound technique strengthen's the idea of backtracking while dealing with Optimization problems. Backtracking applies the depth first search manners to construct the state space tree to obtain the optimal solution, whereas branch & bound follows, breath first search methods.

→ Compared to backtracking, branch and bound requires two additional items,

* A bound value for every node
* Value of the best solution so far

→ If the bound value is not better than the value of the best soln so far, the node (of the bound value), is non-promising and can be pruned

→ In general a search path in the state space tree is terminated for any one of the following reasons,

* Node's bound value not better than the value of the best soln seen so far.
* Node represent No feasible solution.
* The subsets of feasible soln represented by the node has no choices to make.

Scanned by CamScanner

State - Space Tree :

* The root of the state space tree represents the initial state before the search for a solution begins.
* The nodes of the first level represents the choices made for the 1st component of the soln, the nodes of the second level represents the choices made for the 2nd component., this goes on.
* Promising node : A node in the state space tree is a promising node if it corresponds to be a part of the partially constructed solution that may also lead to a complete solution. Otherwise it is known as non-promising node.
* Leaves represent either non-promising dead ends or complete solution found for the problem.
* Constructing the state-space tree :
  - State space tree for backtracking algorithm is constructed in a depth-first manner.
  - If the current node is promising, its child is generated by adding the 1st remaining legitimate option for the next component of the solution, and the processing moves to the child node (i.e) the child node becomes the current node).

Scanned by CamScanner

APPROXIMATION ALGORITHMS FOR NP-HARD PROBLEMS:

→ Travelling salesperson problem and knapsack problem are difficult combinatorial problems when it comes to Optimization.

→ The Optimization version of such difficult combinatorial problems fall in the class of NP-hard Problems. (The decision version of these problems are NP-Complete).

→ Hence for these difficult optimization problems, there is no polynomial time algorithms.

→ Thus, we can deal with a different way for these problems: SOLVE THEM APPROXIMATELY WITH FAST ALGORITHMS.

→ This approach is appealing when applications are necessarily satisfied with approximate solns.

→ Most of the approximation algorithms are problem-specific HEURISTIC.

→ HEURISTIC - It is a common-sense rule drawn from experience rather than from a mathematically proved assertion.

Approximation algorithm for TSP:

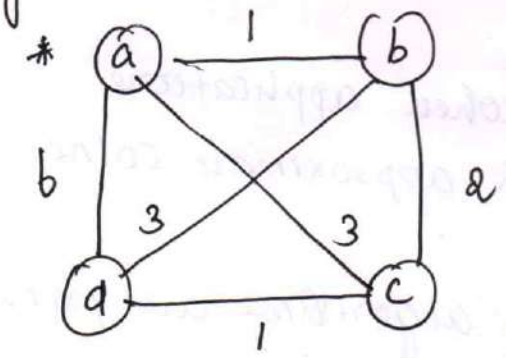
There are three algorithms for TSP, they are

→ Nearest- Neighbour Algorithm
→ Multi fragment - heuristic algorithm
→ Twice around the tree Algorithm (MST based)

Nearest Neighbour Algorithm:

STEP 1: Choose an arbitrary city as start
STEP 2: Repeat the following ~~step~~ op until all cities are visited : go to the nearest unvisited city of the one visited last
STEP 3: Return to start city.

Eg:



Consider 'a' as start vertex,

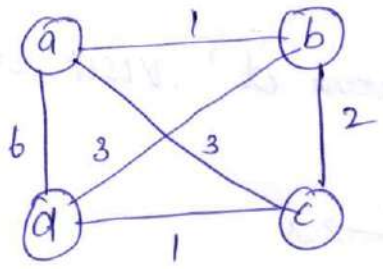
→ 'a' nearest unvisited neighbour is 'b'

$a \xrightarrow{1} b$

→ 'b' nearest unvisited Neighbour is 'c'

$a \xrightarrow{1} b \xrightarrow{2} c$

Eg:



→ Sorted edges List { (a,b), (d,c), (b,c), (a,c), (b,d), (d,a) }

→ tour Edges { ϕ }

Add (a,b) to tour Edges No Issues

{ (a,b) }

a — b

Add (d,c) to tour Edges No Issues

{ (a,b), (d,c) }

a — b
d — c

Add (b,c) to tour Edges No Issues

{ (a,b), (d,c), (b,c) }

a — b
|
d — c

Add (a,c) to tour Edges - Cycle of length '3' is formed which is less than '5', so skip this edge.

a — b
| \
d — c X

Add (b,d) to tour Edge Cycle of length '3' is formed so skip.

a — b
| /
d — c X

→ 'c' nearest unvisited neighbour 'd'

$a \xrightarrow{1} b \xrightarrow{2} c \xrightarrow{1} d$

Since all cities are visited from 'd' visit 'a'

Soln: $a \xrightarrow{1} b \xrightarrow{2} c \xrightarrow{1} d \xrightarrow{6} a$

tour cost = 1+2+1+6 = 10

But the shortest tour is $a \xrightarrow{1} b \xrightarrow{3} d \xrightarrow{1} c \xrightarrow{3} a = 8$

→ The Soln we derived is approximate & is nearer to the Orginal Optimal Soln.

MULTIFRAGMENTED ALGORITHM:

STEP 1: Sort the edges in increasing order of weights

STEP 2: Repeat this step until a tour of length 'n' is obtained: add the next edge on the sorted edge list to the set of tour edges, provided this addition does not create,

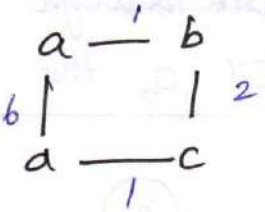
→ a vertex of degree 3

→ a cycle of length less than 'n'.

If so, skip the edge.

STEP 3: Return the set of edges.

Add (d,a) to tour Edges No Issues



four edges { (a,b), (d,c), (b,c), (d,a) }

| Tour Cost = 10 |

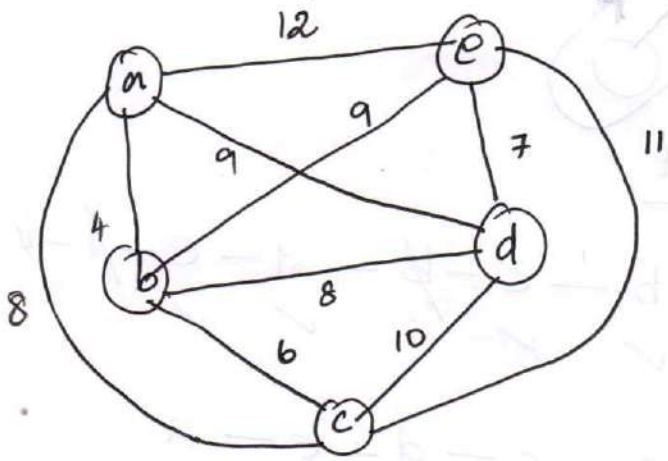
TWICE-AROUND-THE-TREE ALGORITHM:

STEP1: Construct a minimum spanning tree of the graph corresponding to the instance of TSP.

STEP2: Starting from the source vertex, perform a walk around the MST recording all the vertices passed by. (DFS traversal)

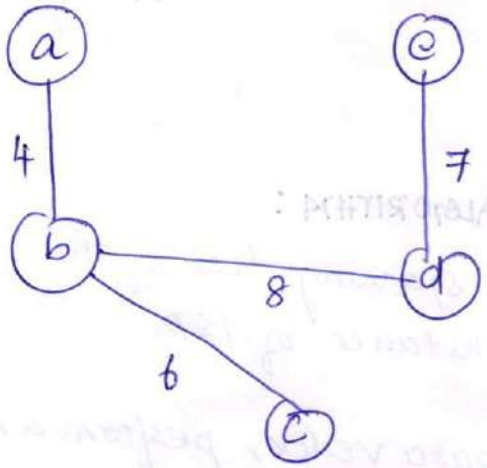
STEP3: Scan the vertex list obtained in step 2, eliminate all repeat occurrences of the a vertex except the start vertex. The vertices remaining on the list forms the Hamiltonian circuit.

Eg:

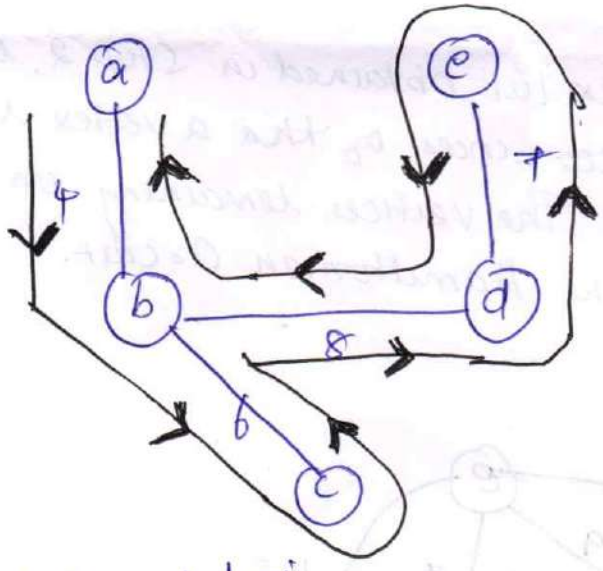


~~Walk through~~

MST of the Graph:



Walk ~~through~~ on the MST:



Walk through list,

a — b — c — ~~b~~ — d — e — ~~d~~ — ~~b~~ — a

✓ ✓ ✓ ✓ ✓ ✓

Result: a — b — c — d — e — a.

P,NP,NP Complete problems

Unit-V

P,NP,NP Complete problems

DEFINITION 1

We say that an algorithm solves a problem in polynomial time if its worst-case time efficiency belongs to O(p(n)) where p(n) is a polynomial of the problem's input size n. (Note that since we are using big-oh notation here, problems solvable in, say, logarithmic time are solvable in polynomial time as well.) Problems that can be solved in polynomial time are called **tractable,** and problems that cannot be solved in polynomial time are called **intractable.**

N,NP Problems

DEFINITION 2 :

Class P is a class of decision problems that can be solved in polynomial time by (deterministic) algorithms. This class of problems is called polynomial.

It is natural to wonder whether every decision problem can be solved in polynomial time. The answer to this question turns out to be no. In fact, some decision problems cannot be solved at all by any algorithm. Such problems are called **undecidable,** as opposed to decidable problems that can be solved by an algorithm.

$$A(P, I) = \begin{cases} 1, & \text{if program } P \text{ halts on input } I; \\ 0, & \text{if program } P \text{ does not halt on input } I. \end{cases}$$

We can consider program P as an input to itself and use the output of algorithm A for pair (P, P) to construct a program Q as follows:

$$Q(P) = \begin{cases} \text{halts}, & \text{if } A(P, P) = 0, \text{ i.e., if program } P \text{ does not halt on input } P; \\ \text{does not halt}, & \text{if } A(P, P) = 1, \text{ i.e., if program } P \text{ halts on input } P. \end{cases}$$

Then on substituting Q for P, we obtain

$$Q(Q) = \begin{cases} \text{halts}, & \text{if } A(Q, Q) = 0, \text{ i.e., if program } Q \text{ does not halt on input } Q; \\ \text{does not halt}, & \text{if } A(Q, Q) = 1, \text{ i.e., if program } Q \text{ halts on input } Q. \end{cases}$$

Here is just a small sample of some of the best-known problems that fall into this category:

Hamiltonian circuit problem :

Determine whether a given graph has a Hamiltonian circuit—a path that starts and ends at the same vertex and passes through all the other vertices exactly once.

Traveling salesman problem :

Find the shortest tour through n cities with known positive integer distances between them (find the shortest Hamiltonian circuit in a complete graph with positive integer weights).

Knapsack problem :

Find the most valuable subset of n items of given positive integer weights and values that fit into a knapsack of a given positive integer capacity.

Partition problem : Given n positive integers, determine whether it is possible to partition them into two disjoint subsets with the same sum.

Bin-packing problem : Given n items whose sizes are positive rational numbers not larger than 1, put them into the smallest number of bins of size 1.

Graph-coloring problem:

For a given graph, find its chromatic number, which is the smallest number of colors that need to be assigned to the graph's vertices so that no two adjacent vertices are assigned the same color.

Integer linear programming problem:

Find the maximum (or minimum) value of a linear function of several integer-valued variables subject to a finite set of constraints in the form of linear equalities and inequalities.

DEFINITION 3

A nondeterministic algorithm is a two-stage procedure that takes as its input an instance I of a decision problem and does the following. Nondeterministic (“guessing”) stage: An arbitrary string S is generated that can be thought of as a candidate solution to the given instance I (but may be complete gibberish as well).

P, NP, and NP-Complete Problems Deterministic **(“verification”) stage:** A deterministic algorithm takes both I and S as its input and outputs yes if S represents a solution to instance I. (If S is not a solution to instance I , the algorithm either returns no or is allowed not to halt at all.)

DEFINITION 4 :

Class NP is the class of decision problems that can be solved by nondeterministic polynomial algorithms. This class of problems is called nondeterministic polynomial. Most decision problems are in NP. First of all, this class includes all the problems in P: $P \subseteq NP$.

This leads to the most important open question of theoretical computer science: Is P a proper subset of NP, or are these two classes, in fact, the same? We can put this symbolically as P ? = NP

Informally, an NP-complete problem is a problem in NP that is as difficult as any other problem in this class because, by definition, any other problem in NP can be reduced to it in polynomial

DEFINITION 5:

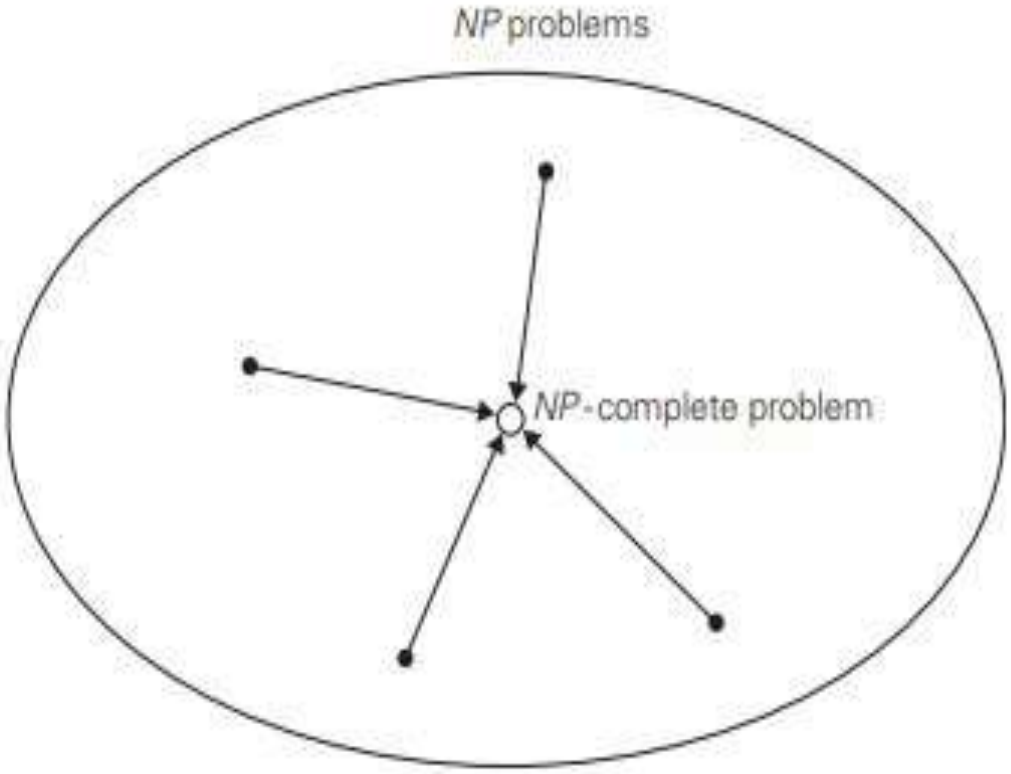
A decision problem D1 is said to be polynomially reducible to a decision problem D2, if there exists a function t that transforms instances of D1 to instances of D2 such that: 1. t maps all yes instances of D1 to yes instances of D2 and all no instances of D1 to no instances of D2 2. t is computable by a polynomial time algorithm This definition immediately implies that if a problem D1 is polynomially reducible to some problem D2 that can be solved in polynomial time, then problem D1 can also be solved in polynomial time (why?).

NP Complete Problems

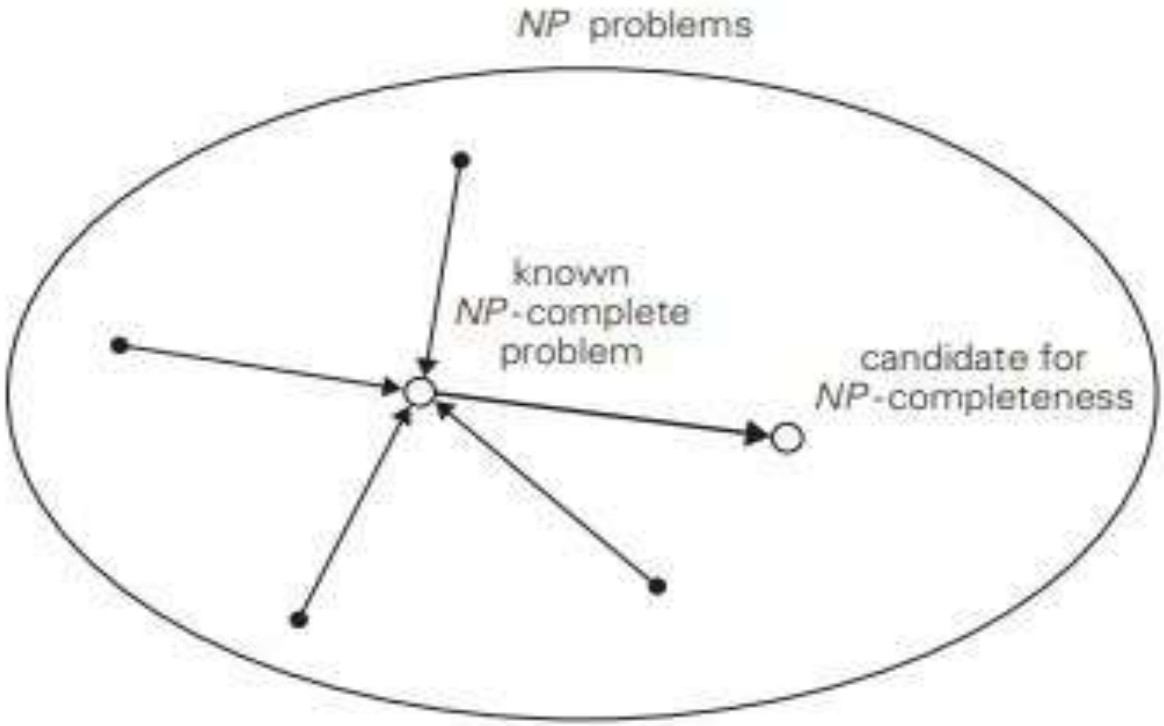
Informally, an NP-complete problem is a problem in NP that is as difficult as any other problem in this class because, by definition, any other problem in NP can be reduced to it in polynomial time Here are more formal definitions of these concepts.

DEFINITION 6 :

A decision problem D is said to be NP-complete if: 1. it belongs to class NP 2. every problem in NP is polynomially reducible to D The fact that closely related decision problems are polynomially reducible to each other is not very surprising. For example, let us prove that the Hamiltonian circuit problem is polynomially reducible to the decision version of the traveling



Notion of an *NP*-complete problem. Polynomial-time reductions of *NP* problems to an *NP*-complete problem are shown by arrows.



Proving *NP*-completeness by reduction.

Whatever the eventual answer to the **P ? = NP** question proves to be, knowing that a problem is NP-complete has important practical implications for today. It means that faced with a problem known to be NP-complete, we should probably not aim at gaining fame and fortune3 by designing a polynomial-time algorithm for solving all its instances